Modelo Lógico — Petly Club

Projeto ERP · Grupo 7 · Modelagem de Banco de Dados — UNICID 2026/2

Base: DER da documentação completa, versão revisada após a devolutiva (16 entidades, RN01–RN25, RP01–RP11).

0. O que esta parte entrega

O DER foi transformado em 16 tabelas, com colunas, tipos, chaves primárias, chaves estrangeiras (inclusive compostas), NOT NULL, UNIQUE, CHECK e DEFAULT. Cada restrição traz a regra de negócio que protege. O que um CHECK não alcança (regra que olha outra tabela, o tempo ou a soma de várias linhas) está marcado como [TRG-nn] e detalhado na seção 6.

Legenda: PK chave primária · FK chave estrangeira · UK única · NN não nulo · CK CHECK · DF DEFAULT · [A1]–[A3] ajuste ao DER aprovado pelo grupo (seção 0.1).

0.1 Ajustes ao DER aprovados pelo grupo

Ao cruzar cada regra de negócio com o DER, três pontos não podiam ser protegidos com o que está desenhado. Foram resolvidos no modelo lógico e estão identificados como [A1], [A2] e [A3]:

- **[A1] Uma única coluna id_produto em movimentacao_estoque, compartilhada pelas três FKs compostas.** O DER já lista id_pedido + id_produto e id_compra + id_produto, mas uma movimentação tem um produto só. A mesma coluna alimenta (id_lote, id_produto) → lote, (id_pedido, id_produto) → item_pedido e (id_compra, id_produto) → item_compra. Efeito: é impossível registrar a venda de um item saindo do lote de outro produto (RN20). A dependência id_lote → id_produto é redundância intencional; avisar quem fizer a normalização.
- **[A2] Coluna id_autorizador em movimentacao_estoque.** RP11 e RF35 exigem autorização da gerência para perdas e ajustes (o cenário 3 diz “autorizada pela gerência”), mas o DER só guarda o funcionário responsável. Sem a coluna, a regra não é auditável.
- **[A3] Colunas id_autorizador_desconto e id_autorizador_cancelamento em pedido.** Mesma lacuna em RP05, RP07, RF16 e RF32. O valor do limite de desconto não existe no modelo; fica como parâmetro da aplicação.

O grupo aprovou os três ajustes. O DER e a documentação da primeira entrega passam a incluir a coluna única id_produto na movimentação [A1] e as colunas de autorização [A2] e [A3].

1. Convenções

- Tabela = entidade do DER em minúsculas e sem acento (cliente, item_pedido, movimentacao_estoque). Colunas em snake_case, como na seção 12 da documentação.
- PK substituta INTEGER GENERATED ALWAYS AS IDENTITY (BIGINT em movimentacao_estoque, que cresce a cada operação). As associativas item_pedido e item_compra usam PK composta, como no DER.

- Tipos: dinheiro NUMERIC(12,2) (NUMERIC(10,2) para preço e custo unitário); quantidades INTEGER; datas DATE; momentos TIMESTAMP; CPF CHAR(11), CNPJ CHAR(14) e CEP CHAR(8) só com dígitos.
- Listas fechadas (status, tipo, canal, forma de pagamento) = CHECK ... IN (...), com valores em minúsculas, sem acento e com sublinhado (loja_fisica, paga_parcialmente).
- Toda FK usa ON DELETE RESTRICT e ON UPDATE NO ACTION. Pedido, compra, parcela e movimentação nunca são apagados: cancelam-se por status (RNF03 exige histórico por 5 anos).
- Atributos derivados do DER não viram coluna: saldo_fisico, quantidade_reservada e quantidade_disponivel (lote), quantidade_recebida (item_compra), valor_pago (conta_pagar), valor_total (compra e pedido) e subtotal (item_pedido). Saem de visões (seção 7).
- SGBD adotado pelo grupo: PostgreSQL 16. Os exemplos, gatilhos e visões deste documento usam a sintaxe do PostgreSQL, incluindo os gatilhos diferidos de [TRG-09] e [TRG-10].

2. Como cada relacionamento do DER virou tabela ou coluna

#	Relacionamento (DER)	Cardinalidade	Implementação lógica	Regra
1	Cliente — realiza — Pedido	(0,N) – (1,1)	pedido.id_cliente, FK NN	RN01, RN02, RP02
2	Funcionário — registra — Pedido	(0,N) – (0,1)	pedido.id_funcionario, FK que aceita nulo; CK exige na loja física	RN14
3	Pedido — contém — Produto	N:N	tabela item_pedido, PK (id_pedido, id_produto), duas FKs	RN03, RN13
4	Pedido — possui — Pagamento	(1,N) – (1,1)	pagamento.id_pedido, FK NN; mínimo de 1 pagamento por [TRG-10]	RN15
5	Pagamento — gera — Conta_Receber	(0,N) – (1,1)	conta_receber.id_pagamento, FK NN	RN24
6	Pedido — gera — Entrega	(0,1) – (1,1)	entrega.id_pedido, FK NN e UK	RN17
7	Categoria — classifica — Produto	(0,N) – (1,1)	produto.id_categoria, FK NN	RN04
8	Produto — possui — Lote	(0,N) – (1,1)	lote.id_produto, FK NN	RP01, RN11
9	Lote — sofre — Movimentação	(1,N) – (1,1)	FK composta (id_lote, id_produto) → lote	RN19
10	Item_Pedido — origina — Movimentação	(1,N) – (0,1)	FK composta (id_pedido, id_produto) → item_pedido, verificada só quando id_pedido está preenchido	RN20–RN22

#	Relacionamento (DER)	Cardinalidade	Implementação lógica	Regra
11	Item_Compra — origina — Movimentação	(0,N) – (0,1)	FK composta (id_compra, id_produto) → item_compra, verificada só quando id_compra está preenchido	RN08, RN25
12	Funcionário — registra — Movimentação	(0,N) – (0,1)	movimentacao_estoque.id_funcionario, FK que aceita nulo; [A2] id_autorizador, FK	RN19, RP06, RP11
13	Fornecedor — atende — Compra	(0,N) – (1,1)	compra.id_fornecedor, FK NN	RN10
14	Compra — repõe — Produto	N:N	tabela item_compra, PK (id_compra, id_produto), duas FKs	RN10, RN25
15	Fornecedor — fornece — Produto	N:N	sem tabela: caminho fornecedor → compra → item_compra ← produto	RN09
16	Compra — gera — Conta_Pagar	(1,N) – (1,1)	conta_pagar.id_compra, FK NN	RN12
17	Conta_Pagar — quita — Pagamento_Conta	(0,N) – (1,1)	pagamento_conta.id_conta_pagar, FK NN	RN23

Os mínimos 1 do lado do pai (pedido com ao menos um item e um pagamento, compra com ao menos um item e uma conta a pagar, lote com ao menos uma entrada, item de pedido com ao menos uma movimentação) não cabem em NOT NULL, porque o filho é que aponta para o pai. Ficam em [TRG-10].

3. Cadastros básicos e estoque

3.1 cliente

Coluna	Tipo	Chave	Restrições	Regra que protege
id_cliente	INTEGER identity	PK	—	Identificação única e imutável, independente do CPF
nome	VARCHAR(120)		NN	RF03: cadastro completo
cpf	CHAR(11)	UK	NN; CK `cpf ~ '^[0-9]{11}$'`	RN18: CPF não repete. Guardar só dígitos impede que 123.456.789-09 e 12345678909 virem dois cadastros. Dígito verificador: aplicação
telefone	VARCHAR(20)		NULL	Dicionário: pode ser informado depois
email	VARCHAR(120)		NULL; CK `email LIKE`	Obrigatório só para compra na loja virtual → [TRG-07]. Sem UK: nenhuma regra

Coluna	Tipo	Chave	Restrições	Regra que protege
			`'%_@_%'`	proíbe e-mail repetido

3.2 funcionario

Coluna	Tipo	Chave	Restrições	Regra que protege
id_funcionario	INTEGER identity	PK	—	Identificação única
nome	VARCHAR(120)		NN	RF05
cpf	CHAR(11)	UK	NN; CK 11 dígitos	Dicionário: não se repete. Sem UK cruzada com cliente: o funcionário que compra é cadastrado também como cliente (seção 17)
cargo	VARCHAR(30)		NN; CK IN ('gerente','atendente','operador_caixa','estoquista','compras','financeiro','loja_virtual')	RP08, RF05, RNF01: o cargo define o acesso. Lista tirada dos processos 1 a 12; o dicionário diz “Ex.”, então novo cargo entra alterando o CK
login	VARCHAR(50)	UK	NN	Dicionário: não se repete; identifica quem acessa
senha	VARCHAR(255)		NN; CK `CHAR_LENGTH(senha) >= 60`	RNF02: só hash. bcrypt e argon2 geram 60 caracteres ou mais; é barreira mínima contra texto puro, e o hash é gerado pela aplicação

3.3 categoria

Coluna	Tipo	Chave	Restrições	Regra que protege
id_categoria	INTEGER identity	PK	—	Identificação única
nome	VARCHAR(60)	UK	NN	Dicionário: não se repete; evita duas categorias iguais nos relatórios (RF31)
descricao	VARCHAR(255)		NULL	Dicionário: opcional

3.4 produto

Coluna	Tipo	Chave	Restrições	Regra que protege
id_produto	INTEGER identity	PK	—	Identificação única
nome	VARCHAR(120)		NN	RF01
descricao	VARCHAR(255)		NULL	Dicionário: opcional
marca	VARCHAR(60)		NN	Dicionário: obrigatória
codigo_barras	VARCHAR(14)	UK	NN; CK `codigo_barras ~ '^[0-9]{8,14}$'`	RF01; fluxograma 10 (barras já cadastrado = não cadastrar); RNF05: o índice da UK sustenta a busca em até 1 s
preco	NUMERIC(10,2)		NN; CK `preco > 0`	Dicionário: maior que zero. RP04 (só a gerência altera) é permissão, não CHECK → [TRG-17]. RN13: o preço vendido fica em item_pedido
estoque_minimo	INTEGER		NN; DF 0; CK `estoque_minimo >= 0`	RP03, RF13: base do alerta de reposição
controla_validade	BOOLEAN		NN, sem DEFAULT	RF02, RN11. Sem DEFAULT de propósito: o cadastro é obrigado a decidir. DEFAULT FALSE deixaria uma ração entrar sem validade e furaria RP09
id_categoria	INTEGER	FK→categoria	NN	RN04: todo produto tem categoria

3.5 lote (antiga entidade Estoque)

Coluna	Tipo	Chave	Restrições	Regra que protege
id_lote	INTEGER identity	PK	—	Identificação única
id_produto	INTEGER	FK→produto	NN	RP01, RN11: todo lote pertence a um produto
codigo_lote	VARCHAR(40)		NULL	RN11: obrigatório quando o produto controla validade [TRG-05]
data_validade	DATE		NULL	RN11, RP09; vencimento verificado em [TRG-03]
data_entrada	DATE		NN; DF CURRENT_DATE	Dicionário: obrigatória

Restrições de tabela:

- `uk_lote_produto_codigo` UNIQUE (id_produto, codigo_lote): o mesmo código do mesmo produto não gera dois lotes. Uma segunda entrega do mesmo lote reaproveita o registro e soma uma movimentação de entrada. Vários nulos são aceitos (produtos sem controle de validade).
- `uk_lote_id_produto` UNIQUE (id_lote, id_produto): alvo da FK composta de movimentacao_estoque [A1].
- `ck_lote_codigo`: `data_validade IS NULL OR codigo_lote IS NOT NULL`. RN11: validade sem código de lote não identifica nada.
- `ck_lote_validade`: `data_validade IS NULL OR data_validade >= data_entrada`. RP09: lote já vencido não entra no estoque.
- Sem colunas de saldo: saldo_fisico, quantidade_reservada e quantidade_disponivel vêm de v_lote_saldo (RN19), e o saldo físico nunca fica negativo por [TRG-02] (dicionário).

3.6 movimentacao_estoque (histórico imutável do estoque)

Coluna	Tipo	Chave	Restrições	Regra que protege
id_movimentacao	BIGINT identity	PK	—	RN19: cada alteração de quantidade é um registro próprio
id_lote	INTEGER	FK composta	NN	RN19, RN20: toda movimentação pertence a um lote
id_produto	INTEGER	FK composta (3 FKs)	NN	[A1] garante que lote, item do

Coluna	Tipo	Chave	Restrições	Regra que protege
				pedido e item da compra são do mesmo produto
tipo	VARCHAR(20)		NN; CK IN ('entrada','reserva','liberacao_reserva','venda','devolucao','perda','ajuste_positivo','ajuste_negativo')	RN19: os 8 tipos. O tipo decide se soma ou subtrai
quantidade	INTEGER		NN; CK `quantidade > 0`	Dicionário: maior que zero. O sinal vem do tipo, então nunca é negativa
data_hora	TIMESTAMP		NN; DF CURRENT_TIMESTAMP	RN19, RNF03: preenchida pelo sistema, não pelo operador
motivo	VARCHAR(255)		NULL; ver ck_mov_motivo	RP11, RF35
id_funcionario	INTEGER	FK→funcionario	NULL; ver ck_mov_responsavel	RN19: responsável
id_autorizador [A2]	INTEGER	FK→funcionario	NULL; ver ck_mov_autorizacao	RP11, RF35
id_pedido	INTEGER	FK composta	NULL; ver ck_mov_vinculo	RN20–RN22
id_compra	INTEGER	FK composta	NULL; ver ck_mov_vinculo	RN25

Chaves estrangeiras: (id_lote, id_produto) → lote(id_lote, id_produto); (id_pedido, id_produto) → item_pedido; (id_compra, id_produto) → item_compra; id_funcionario e id_autorizador → funcionario. Como id_produto é sempre preenchido, a FK do item só é verificada quando id_pedido (ou id_compra) existe, e as de perda e ajuste ficam livres de vínculo.

Restrições de tabela:

Nome	Condição	Regra que protege
ck_mov_vinculo	`(tipo = 'entrada' AND id_compra IS NOT NULL AND id_pedido IS NULL) OR (tipo IN ('reserva','liberacao_reserva','venda','devolucao') AND id_pedido IS NOT NULL AND id_compra IS NULL) OR (tipo IN ('perda','ajuste_positivo','ajuste_negativo') AND id_pedido IS NULL AND id_compra IS NULL)`	Entrada nasce de item de compra (RN25). Reserva, liberação, venda e devolução nascem de item de pedido (RN20–RN22). Perda e ajuste não têm item (seção 14 do DER). Nenhuma movimentação carrega os dois vínculos
ck_mov_motivo	`tipo NOT IN ('devolucao','perda','ajuste_positivo','ajuste_negativo') OR (motivo IS NOT NULL AND CHAR_LENGTH(TRIM(motivo)) > 0)`	RP11, RF35 e dicionário: motivo obrigatório em perdas, ajustes e devoluções
ck_mov_responsavel	`tipo IN ('reserva','liberacao_reserva') OR id_funcionario IS NOT NULL`	RN19: toda movimentação tem responsável, exceto reserva e liberação automáticas da loja virtual (seção 14). Cobre também RP06: entrada sempre tem funcionário
ck_mov_autorizacao [A2]	`tipo NOT IN ('perda','ajuste_positivo','ajuste_negativo') OR id_autorizador IS NOT NULL`	RP11, RF35: perda e ajuste só com autorização. Que o autorizador seja gerente → [TRG-17]

A tabela é imutável: nenhum UPDATE nem DELETE ([TRG-01]). Erro se corrige com novo ajuste (RN19, RNF03).

4. Fornecedores, compras e contas a pagar

4.1 fornecedor

Coluna	Tipo	Chave	Restrições	Regra que protege
id_fornecedor	INTEGER identity	PK	—	Identificação única
razao_social	VARCHAR(150)		NN	Dicionário: obrigatória
cnpj	CHAR(14)	UK	NN; CK 14	Dicionário: não se repete; mesma

Coluna	Tipo	Chave	Restrições	Regra que protege
			dígitos	lógica de formato do CPF
telefone	VARCHAR(20)		NN	Dicionário: obrigatório
email	VARCHAR(120)		NULL	Dicionário: opcional

4.2 compra

Coluna	Tipo	Chave	Restrições	Regra que protege
id_compra	INTEGER identity	PK	—	Identificação única
id_fornecedor	INTEGER	FK→fornecedor	NN	RN10: toda compra tem fornecedor
data_compra	DATE		NN; DF CURRENT_DATE	Dicionário: obrigatória
status	VARCHAR(25)		NN; DF 'solicitada'; CK IN ('solicitada','recebida_parcialmente','recebida','cancelada')	RN25: a compra só é recebida quando todos os itens chegam; mantido por [TRG-06]

valor_total é derivado (soma de quantidade_comprada × custo_unitario): visão v_compra_total.

4.3 item_compra (resolve Compra N:N Produto)

Coluna	Tipo	Chave	Restrições	Regra que protege
id_compra	INTEGER	PK + FK→compra	NN	RN10
id_produto	INTEGER	PK + FK→produto	NN	RN10: PK composta = o produto aparece uma vez por compra, com a quantidade total
quantidade_comprada	INTEGER		NN; CK `quantidade_comprada > 0`	Dicionário: maior que zero
custo_unitario	NUMERIC(10,2)		NN; CK `custo_unitario > 0`	RN10: custo daquela compra, preserva o histórico e permite

Coluna	Tipo	Chave	Restrições	Regra que protege
				margem

quantidade_recebida é derivada das movimentações de entrada do item (RN25): visão v_item_compra_recebido. Lote e validade ficam em lote, não aqui (RN11).

4.4 conta_pagar

Coluna	Tipo	Chave	Restrições	Regra que protege
id_conta_pagar	INTEGER identity	PK	—	Identificação única
id_compra	INTEGER	FK→compra	NN	RN12: toda parcela vem de uma compra
numero_parcela	SMALLINT		NN; CK `numero_parcela >= 1`	Dicionário: ex. 1 de 3
valor	NUMERIC(12,2)		NN; CK `valor > 0`	Dicionário: maior que zero
data_vencimento	DATE		NN	Dicionário: obrigatória; base do fluxo de caixa (RF37)
status	VARCHAR(20)		NN; DF 'pendente'; CK IN ('pendente','paga_parcialmente','paga','cancelada')	RN23: só fica paga quando a soma dos pagamentos atinge o valor; mantido por [TRG-08]

Restrição de tabela: `uk_conta_pagar_parcela` UNIQUE (id_compra, numero_parcela): a parcela 2 de uma compra existe uma vez só (RN12). valor_pago é derivado: visão v_conta_pagar_saldo.

4.5 pagamento_conta

Coluna	Tipo	Chave	Restrições	Regra que protege
id_pagamento_conta	INTEGER identity	PK	—	Identificação única
id_conta_pagar	INTEGER	FK→conta_pagar	NN	RN23: todo pagamento quita uma parcela
valor_pago	NUMERIC(12,2)		NN; CK `valor_pago >`	Dicionário: maior que zero. A soma

Coluna	Tipo	Chave	Restrições	Regra que protege
			0	não passar do valor da parcela → [TRG-08]
data_pagamento	DATE		NN; DF CURRENT_DATE	Dicionário: obrigatória
forma_pagamento	VARCHAR(20)		NN; CK IN ('pix','boleto','transferencia')	Dicionário (“Ex.: Pix, boleto, transferência”); ampliar alterando o CK

5. Vendas, pagamentos e entregas

5.1 pedido

Coluna	Tipo	Chave	Restrições	Regra que protege
id_pedido	INTEGER identity	PK	—	Dicionário: não muda ao longo do tempo
id_cliente	INTEGER	FK→cliente	NN	RN01, RP02: todo pedido tem cliente cadastrado, inclusive no caixa
id_funcionario	INTEGER	FK→funcionario	NULL; ver ck_pedido_func_fisica	RN14: venda física tem funcionário; a loja virtual não tem atendente
data_pedido	TIMESTAMP		NN; DF CURRENT_TIMESTAMP	Dicionário: obrigatória
canal_venda	VARCHAR(12)		NN; CK IN ('loja_fisica','loja_virtual')	RN05: todo pedido tem canal
tipo_recebimento	VARCHAR(10)		NN; CK IN ('balcao','retirada'	RN17

Coluna	Tipo	Chave	Restrições	Regra que protege
			,'entrega')	
status	VARCHAR(15)		NN; DF 'pendente'; CK IN ('pendente','pago','em_preparacao','enviado','entregue','retirado','cancelado')	RN16: todo pedido tem status; nasce pendente
desconto	NUMERIC(10,2)		NN; DF 0; CK `desconto >= 0`	RP05: pode ser zero, nunca negativo
id_autorizador_desconto [A3]	INTEGER	FK→funcionario	NULL	RP05, RF32: quem autorizou desconto acima do limite
id_autorizador_cancelamento [A3]	INTEGER	FK→funcionario	NULL; ver ck_pedido_cancel_fisica	RP07, RF16: quem autorizou o cancelamento no caixa

Restrições de tabela:

Nome	Condição	Regra que protege
ck_pedido_canal_recebimento	`(canal_venda = 'loja_fisica' AND tipo_recebimento = 'balcao') OR (canal_venda = 'loja_virtual' AND tipo_recebimento IN ('retirada','entrega'))`	RN17: só a loja virtual tem retirada ou entrega; o caixa é balcão
ck_pedido_func_fisica	`canal_venda <> 'loja_fisica' OR id_funcionario IS NOT NULL`	RN14
ck_pedido_status_fluxo	`(status NOT IN ('em_preparacao','enviado','entregue','retirado') OR canal_venda = 'loja_virtual') AND (status NOT IN ('enviado','entregue') OR tipo_recebimento = 'entrega') AND (status <> 'retirado' OR tipo_recebimento = 'retirada')`	RN16, RN17: o status acompanha o tipo de recebimento (não existe pedido de balcão “entregue”)
ck_pedido_cancel_fisica [A3]	`NOT (canal_venda = 'loja_fisica'`	RP07, RF16:

Nome	Condição	Regra que protege
	`AND status = 'cancelado') OR id_autorizador_cancelamento IS NOT NULL`	cancelar na loja física exige autorização da gerência

valor_total é derivado (soma dos subtotais menos o desconto): visão v_pedido_total. Regras de transição e de limite em [TRG-14] e [TRG-16].

5.2 item_pedido (resolve Pedido N:N Produto)

Coluna	Tipo	Chave	Restrições	Regra que protege
id_pedido	INTEGER	PK + FK→pedido	NN	RN03
id_produto	INTEGER	PK + FK→produto	NN	RN03: PK composta = o mesmo produto não se repete no pedido
quantidade	INTEGER		NN; CK `quantidade > 0`	RN07: não pode superar o disponível [TRG-02]; o lote de saída fica nas movimentações (RN20)
preco_unitario	NUMERIC(10,2)		NN; CK `preco_unitario > 0`	RN13: preço do momento da venda, copiado de produto.preco por [TRG-13]

subtotal é derivado (quantidade × preco_unitario). Não existe coluna de lote aqui: um item pode sair de dois lotes (cenário 2).

5.3 pagamento

Coluna	Tipo	Chave	Restrições	Regra que protege
id_pagamento	INTEGER identity	PK	—	Identificação única
id_pedido	INTEGER	FK→pedido	NN	RN15: todo pagamento refere-se a um pedido; vários por pedido
forma_pagamento	VARCHAR(15)		NN; CK IN ('dinheiro','pix','cartao_debito','cart	RF15; ‘dinheiro’ só na loja física em [TRG-11]

Coluna	Tipo	Chave	Restrições	Regra que protege
			ao_credito')	
valor	NUMERIC(12,2)		NN; CK `valor > 0`	Dicionário: maior que zero
numero_parcelas	SMALLINT		NN; DF 1; CK `numero_parcelas BETWEEN 1 AND 3`	RN24: crédito em até 3x sem juros
data_autorizacao	TIMESTAMP		NULL; ver ck_pag_autorizacao	Dicionário: preenchida quando autorizado
status	VARCHAR(12)		NN; DF 'pendente'; CK IN ('pendente','autorizado','recusado','estornado')	RN15: a loja virtual pode ficar pendente até a confirmação (cenários 6 e 7)

Restrições de tabela:

- `ck_pag_parcelas_credito`: `forma_pagamento = 'cartao_credito' OR numero_parcelas = 1`. RN24 e dicionário: 1 a 3 parcelas só no crédito; nas demais formas, 1.
- `ck_pag_autorizacao`: `(status IN ('autorizado','estornado') AND data_autorizacao IS NOT NULL) OR (status NOT IN ('autorizado','estornado') AND data_autorizacao IS NULL)`. Só tem data de autorização quem foi autorizado (um estornado já foi autorizado antes). Pendente e recusado não têm.

5.4 conta_receber

Coluna	Tipo	Chave	Restrições	Regra que protege
id_conta_receber	INTEGER identity	PK	—	Identificação única
id_pagamento	INTEGER	FK→pagamento	NN	RN24: só pagamento autorizado gera conta [TRG-12]
numero_parcela	SMALLINT		NN; CK `numero_parcela BETWEEN 1 AND 3`	RN24: no máximo 3 parcelas

Coluna	Tipo	Chave	Restrições	Regra que protege
valor_bruto	NUMERIC(12,2)		NN; CK `valor_bruto > 0`	Dicionário: soma das parcelas = valor do pagamento [TRG-12]
valor_liquido	NUMERIC(12,2)		NN; CK `valor_liquido > 0 AND valor_liquido <= valor_bruto`	Dicionário: bruto menos a taxa da operadora; igual ao bruto em dinheiro e Pix (cenário 6: 100,00 bruto, 97,00 líquido)
data_prevista	DATE		NN	Dicionário: obrigatória; base do pendente no fluxo de caixa (RF37)
data_recebimento	DATE		NULL; ver ck_cr_recebimento	RN24: só é recebido quando entra na conta
status	VARCHAR(10)		NN; DF 'pendente'; CK IN ('pendente','recebida','cancelada')	RN24, RN22: cancelada quando o pagamento é estornado

Restrições de tabela:

- `uk_cr_parcela` UNIQUE (id_pagamento, numero_parcela): cada parcela existe uma vez só.
- `ck_cr_recebimento`: `(status = 'pendente' AND data_recebimento IS NULL) OR (status = 'recebida' AND data_recebimento IS NOT NULL) OR status = 'cancelada'`. RN24: pendente não tem data de recebimento e recebida tem. A cancelada aceita as duas situações, porque pode ser estorno de valor que já entrou (cenário 8).

5.5 entrega

O endereço é atributo composto no DER e virou sete colunas atômicas (1FN).

Coluna	Tipo	Chave	Restrições	Regra que protege
id_entrega	INTEGER identity	PK	—	Identificação única

Coluna	Tipo	Chave	Restrições	Regra que protege
id_pedido	INTEGER	FK→pedido, UK	NN	RN17: no máximo uma entrega por pedido (a UK faz o 1:1); só para tipo_recebimento = 'entrega' [TRG-15]
logradouro	VARCHAR(120)		NN	Endereço só existe na entrega (RN17)
numero	VARCHAR(10)		NN	Texto, para aceitar ‘S/N’ e ‘12-A’
complemento	VARCHAR(60)		NULL	Opcional
bairro	VARCHAR(60)		NN	
cidade	VARCHAR(60)		NN	
uf	CHAR(2)		NN; CK `uf ~ '^[A-Z]{2}$'`	Formato da sigla
cep	CHAR(8)		NN; CK `cep ~ '^[0-9]{8}$'`	Só dígitos
data_prevista	DATE		NN	Dicionário: obrigatória
data_entrega	DATE		NULL; ver ck_entrega_data	Dicionário: preenchida ao concluir
status	VARCHAR(15)		NN; DF 'em_separacao'; CK IN ('em_separacao','enviado','entregue')	Dicionário (“Ex.”): acompanha o status do pedido [TRG-15]

Restrição de tabela: `ck_entrega_data`: `(status = 'entregue' AND data_entrega IS NOT NULL) OR (status <> 'entregue' AND data_entrega IS NULL)`. Entrega concluída tem data; as demais não.

6. Regras que o DDL sozinho não garante (gatilhos e transações)

Cada linha diz quando verificar e o que verificar. Em PostgreSQL: triggers BEFORE/AFTER e constraint triggers DEFERRABLE; as travas usam SELECT … FOR UPDATE. Regras marcadas como inferidas não estão escritas na documentação e precisam de confirmação do grupo.

ID	Regra	Quando	O que verifica
TRG-01	RN19, RNF03	BEFORE UPDATE ou DELETE em movimentacao_estoque	Recusa sempre. Correção só por nova movimentação de ajuste. Complemento: REVOKE UPDATE, DELETE dos papéis da aplicação (código na seção 7)
TRG-02	RN07, RF10, RNF10	BEFORE INSERT em movimentacao_estoque	Trava a linha do lote (FOR UPDATE), aplica o efeito da nova movimentação sobre v_lote_saldo e recusa se saldo_fisico < 0, quantidade_reservada < 0 ou quantidade_reservada > saldo_fisico. A trava serializa as duas vendas da última unidade (cenário 1): uma é aceita e a outra recusada
TRG-03	RF12, RP09	BEFORE INSERT (reserva, venda, entrada, devolução)	Recusa se lote.data_validade < CAST(data_hora AS DATE) da própria movimentação. A comparação usa a data da movimentação, e não CURRENT_DATE, para que a carga de movimentações antigas (como a entrada de um lote que só venceu depois) não seja recusada. Lote vencido só admite perda, liberação de reserva e ajuste. Se uma reserva venceu antes do envio, libera-se a reserva e registra-se a perda
TRG-04	RN20, RN21, RN22	BEFORE INSERT em movimentacao_estoque	Reserva só em pedido de canal loja_virtual. Venda em pedido virtual só até o valor reservado em aberto do item naquele lote. Liberação não passa da reserva em aberto. Devolução só em pedido físico e até vendido menos devolvido do item no lote. Total reservado ou vendido do item não passa de item_pedido.quantidade
TRG-05	RN11, RF08	BEFORE INSERT ou UPDATE em lote	Se produto.controla_validade, exige codigo_lote e data_validade
TRG-06	RN25, RF34	AFTER INSERT em movimentacao_estoque (entrada)	Soma das entradas do item não passa de quantidade_comprada (excesso é divergência registrada à parte, fluxograma 7; inferida). Recusa entrada em compra cancelada. Atualiza compra.status: solicitada, recebida_parcialmente, recebida
TRG-07	Dicionário	BEFORE INSERT em	Pedido de canal loja_virtual exige cliente

ID	Regra	Quando	O que verifica
	(cliente.email)	pedido	com e-mail preenchido
TRG-08	RN23, RF36	BEFORE INSERT em pagamento_conta	Trava a parcela (FOR UPDATE). Recusa se a soma dos pagamentos mais o novo passar do valor, ou se a parcela estiver cancelada. Atualiza status para paga_parcialmente ou paga
TRG-09	RN12	No commit (constraint trigger DEFERRABLE) em compra	Soma de conta_pagar.valor da compra = valor_total da compra (inferida de RN12 e do cenário 5)
TRG-10	RN03, RN10, RN12, RN15, RN19, RN21	No commit (DEFERRABLE INITIALLY DEFERRED)	Pedido tem ao menos um item_pedido e um pagamento; compra tem ao menos um item_compra e uma conta_pagar; lote tem ao menos uma movimentação de entrada (Lote (1,N) do DER); item_pedido tem ao menos uma movimentação (reserva na loja virtual, venda na loja física), registrada na mesma transação do item, exceto item de pedido físico cancelado antes da conclusão da venda. Tem de ser no commit porque o filho aponta para o pai
TRG-11	RN15, RN16, RF15	BEFORE INSERT ou UPDATE em pagamento	‘dinheiro’ só em pedido da loja física (processos 1 e 2; inferida). Soma dos pagamentos autorizados não passa do valor_total do pedido; ao igualar, o pedido vira ‘pago’
TRG-12	RN24, RN22	pagamento e conta_receber	Só pagamento ‘autorizado’ gera conta. Quantidade de contas = numero_parcelas. Soma de valor_bruto = valor do pagamento (a última parcela absorve os centavos: 100,00 em 3x = 33,33 + 33,33 + 33,34). Pagamento ‘estornado’ leva as contas a ‘cancelada’
TRG-13	RN13	BEFORE INSERT em item_pedido	preco_unitario recebe produto.preco vigente e não muda depois. Mudar o preço do produto não altera pedidos antigos
TRG-14	RP07, RN22	BEFORE UPDATE de pedido.status	Pedido da loja virtual em enviado, entregue ou retirado não vai para cancelado. Cancelar exige liberação (só reservado) ou devolução

ID	Regra	Quando	O que verifica
			e perda (já baixado) registradas
TRG-15	RN17	BEFORE INSERT em entrega	Pedido tem de ter tipo_recebimento = 'entrega'. Status de pedido e entrega andam juntos
TRG-16	RP05, RF32	BEFORE INSERT ou UPDATE em pedido	desconto não passa da soma dos itens. Acima do limite da empresa (parâmetro da aplicação, não modelado) exige id_autorizador_desconto [A3]
TRG-17	RP04, RP06, RP08, RP11, RNF01, RNF09	Permissões do banco	Papéis por cargo: preço só gerente; entrada só estoquista ou gerente; perda e ajuste exigem id_autorizador com cargo gerente; CPF mascarado por visão para cargos sem permissão
TRG-18	RN20, RF11	Aplicação	Sugere os lotes por data_validade crescente (nulos por último, desempate por data_entrada) e grava uma movimentação de venda por lote usado

7. Visões e DDL das peças críticas

7.1 Saldos do lote (derivados de movimentacao_estoque)

```
CREATE VIEW v_lote_saldo AS
SELECT t.id_lote, t.id_produto, t.saldo_fisico, t.quantidade_reservada,
       t.saldo_fisico - t.quantidade_reservada AS quantidade_disponivel
FROM (
  SELECT l.id_lote, l.id_produto,
    COALESCE(SUM(CASE WHEN m.tipo IN
('entrada','devolucao','ajuste_positivo') THEN m.quantidade
                      WHEN m.tipo IN ('venda','perda','ajuste_negativo') THEN
-m.quantidade
                      ELSE 0 END), 0) AS saldo_fisico,
    COALESCE(SUM(CASE WHEN m.tipo = 'reserva' THEN m.quantidade
                      WHEN m.tipo = 'liberacao_reserva' THEN -m.quantidade
                      WHEN m.tipo = 'venda' AND p.canal_venda =
'loja_virtual' THEN -m.quantidade
                      ELSE 0 END), 0) AS quantidade_reservada
  FROM lote l
  LEFT JOIN movimentacao_estoque m ON m.id_lote = l.id_lote
  LEFT JOIN pedido p ON p.id_pedido = m.id_pedido
  GROUP BY l.id_lote, l.id_produto
) t;
```

A venda de um pedido da loja virtual consome a reserva (RN21): baixa o saldo físico e a reserva na mesma quantidade, então o disponível não muda. A venda no caixa baixa só o saldo físico. Conferindo o

cenário 7: reserva de 2 no lote L-202 dá saldo 9, reservado 2, disponível 7; a liberação volta a reservado 0 e disponível 9.

7.2 Demais visões

- **v_item_compra_recebido**: quantidade_recebida = soma das entradas por (id_compra, id_produto), ao lado de quantidade_comprada (RN25).
- **v_conta_pagar_saldo**: valor_pago_total = soma de pagamento_conta.valor_pago e saldo_devedor = valor menos valor_pago_total. O nome muda para não confundir com a coluna valor_pago de pagamento_conta (RN23).
- **v_pedido_total** e **v_compra_total**: soma de quantidade × preço (ou custo) dos itens, menos o desconto no pedido.
- **v_produto_estoque**: soma do disponível dos lotes não vencidos por produto, comparada a estoque_minimo (RF13, RP03) e usada pelo catálogo da loja virtual (RF17).
- **v_cliente_mascarado**: cpf como CONCAT('*.', SUBSTR(cpf,4,3), '.', SUBSTR(cpf,7,3), '-') para cargos sem permissão (RNF09).

7.3 movimentacao_estoque e a trava de imutabilidade

Ordem de criação das tabelas: categoria, cliente, funcionario, fornecedor, produto, lote, compra, item_compra, pedido, item_pedido, pagamento, conta_receber, entrega, conta_pagar, pagamento_conta e, por último, movimentacao_estoque (depende de lote, item_pedido, item_compra e funcionario).

```
CREATE TABLE movimentacao_estoque (
  id_movimentacao BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_lote         INTEGER     NOT NULL,
  id_produto      INTEGER     NOT NULL,
  tipo            VARCHAR(20) NOT NULL,
  quantidade      INTEGER     NOT NULL,
  data_hora       TIMESTAMP   NOT NULL DEFAULT CURRENT_TIMESTAMP,
  motivo          VARCHAR(255),
  id_funcionario  INTEGER,
  id_autorizador  INTEGER,              -- [A2]
  id_pedido       INTEGER,
  id_compra       INTEGER,
  CONSTRAINT fk_mov_lote        FOREIGN KEY (id_lote, id_produto)
REFERENCES lote (id_lote, id_produto)          ON DELETE RESTRICT,
  CONSTRAINT fk_mov_item_pedido FOREIGN KEY (id_pedido, id_produto)
REFERENCES item_pedido (id_pedido, id_produto) ON DELETE RESTRICT,
  CONSTRAINT fk_mov_item_compra FOREIGN KEY (id_compra, id_produto)
REFERENCES item_compra (id_compra, id_produto) ON DELETE RESTRICT,
  CONSTRAINT fk_mov_funcionario FOREIGN KEY (id_funcionario) REFERENCES
funcionario (id_funcionario) ON DELETE RESTRICT,
  CONSTRAINT fk_mov_autorizador FOREIGN KEY (id_autorizador) REFERENCES
funcionario (id_funcionario) ON DELETE RESTRICT,
  CONSTRAINT ck_mov_tipo CHECK (tipo IN
('entrada','reserva','liberacao_reserva','venda','devolucao','perda','ajuste_
positivo','ajuste_negativo')),
  CONSTRAINT ck_mov_quantidade CHECK (quantidade > 0),
```

```
  CONSTRAINT ck_mov_vinculo CHECK (
       (tipo = 'entrada' AND id_compra IS NOT NULL AND id_pedido IS NULL)
    OR (tipo IN ('reserva','liberacao_reserva','venda','devolucao') AND
id_pedido IS NOT NULL AND id_compra IS NULL)
    OR (tipo IN ('perda','ajuste_positivo','ajuste_negativo') AND id_pedido
IS NULL AND id_compra IS NULL)),
  CONSTRAINT ck_mov_motivo CHECK (tipo NOT IN
('devolucao','perda','ajuste_positivo','ajuste_negativo')
    OR (motivo IS NOT NULL AND CHAR_LENGTH(TRIM(motivo)) > 0)),
  CONSTRAINT ck_mov_responsavel CHECK (tipo IN
('reserva','liberacao_reserva') OR id_funcionario IS NOT NULL),
  CONSTRAINT ck_mov_autorizacao CHECK (tipo NOT IN
('perda','ajuste_positivo','ajuste_negativo') OR id_autorizador IS NOT NULL)
);

CREATE FUNCTION fn_mov_imutavel() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  RAISE EXCEPTION 'Movimentacao de estoque nao pode ser alterada nem excluida
(RN19). Registre um ajuste.';
END; $$;

CREATE TRIGGER trg01_mov_imutavel BEFORE UPDATE OR DELETE ON
movimentacao_estoque
  FOR EACH ROW EXECUTE FUNCTION fn_mov_imutavel();
```

8. Cobertura das regras de negócio

Regra	Onde está protegida
RN01, RN02	pedido.id_cliente NN; cliente sem pedido é permitido (nenhuma restrição no lado do cliente)
RN03	PK composta de item_pedido; mínimo de 1 item em [TRG-10]
RN04	produto.id_categoria NN com FK
RN05	pedido.canal_venda NN com CK
RN06, RN08	Venda e entrada são movimentações; não existe coluna de quantidade em produto ou lote
RN07	[TRG-02] (e CK quantidade > 0)
RN09	Sem tabela: fornecedor → compra → item_compra ← produto
RN10	compra.id_fornecedor NN; item_compra; [TRG-10]
RN11	ck_lote_codigo, [TRG-05]
RN12	conta_pagar.id_compra NN, uk_conta_pagar_parcela, [TRG-09], [TRG-10]
RN13	item_pedido.preco_unitario NN com CK, [TRG-13]
RN14	ck_pedido_func_fisica

Regra	Onde está protegida
RN15	pagamento.id_pedido NN, status com DF pendente, [TRG-10], [TRG-11]
RN16	pedido.status NN com CK e DF, ck_pedido_status_fluxo
RN17	UK em entrega.id_pedido, ck_pedido_canal_recebimento, [TRG-15]
RN18	UK em cliente.cpf com formato só de dígitos
RN19	movimentação: CKs de tipo, quantidade, vínculo, motivo e responsável; DF data_hora; [TRG-01]; saldos pela visão
RN20	FK composta (id_lote, id_produto) e (id_pedido, id_produto); [TRG-04]; [TRG-18]
RN21	[TRG-04] (reserva só virtual, venda consome a reserva); v_lote_saldo
RN22	tipos devolucao e perda; [TRG-04], [TRG-14]
RN23	pagamento_conta, conta_pagar.status, [TRG-08]
RN24	ck_pag_parcelas_credito, CKs de conta_receber, uk_cr_parcela, [TRG-12]
RN25	item_compra sem coluna de recebido; [TRG-06]; compra.status
RP01	Um único par lote e movimentacao_estoque, sem coluna de loja
RP02	pedido.id_cliente NN
RP03	produto.estoque_minimo e v_produto_estoque
RP04, RP08	cargo com CK e [TRG-17]
RP05	[A3] id_autorizador_desconto e [TRG-16]
RP06	ck_mov_responsavel (entrada exige funcionário) e [TRG-17]
RP07	ck_pedido_cancel_fisica [A3] e [TRG-14]
RP09	ck_lote_validade e [TRG-03]
RP10	Ausência proposital de tabelas de serviço, pet e agendamento
RP11	ck_mov_motivo e ck_mov_autorizacao [A2]

9. Divergências e pontos de atenção encontrados na documentação

1. **Autorização sem coluna.** RP05, RP07, RP11, RF16, RF32 e RF35 pedem autorização da gerência, mas o DER não guarda quem autorizou. Resolvido com [A2] e [A3]; o limite de desconto continua sem parâmetro.

2. **Produto repetido nas FKs da movimentação.** A seção 12 lista id_pedido + id_produto e id_compra + id_produto como se fossem colunas diferentes. [A1] usa uma só, e a dependência id_lote → id_produto passa a ser redundância intencional para a etapa de normalização.
3. **Motivo.** A RN19 diz que toda movimentação tem motivo; o dicionário exige só em perdas, ajustes e devoluções. Foi adotado o dicionário, que é mais específico. Para exigir sempre, basta tornar motivo NOT NULL.
4. **Perda por cancelamento sem item de pedido.** O cenário 8 registra a perda da ração rasgada sem ligar ao item do pedido 5060, como manda a seção 14. A ligação com o pedido fica só no motivo e no horário. Se o grupo quiser o vínculo, é só permitir id_pedido em perda no ck_mov_vinculo.
5. **Estados derivados guardados.** compra.status, conta_pagar.status e o status 'pago' do pedido poderiam ser calculados, mas o DER os guarda como atributos. Ficam armazenados e sincronizados por [TRG-06], [TRG-08] e [TRG-11].
6. **Mesmo relacionamento, duas direções.** A seção 13 escreve PAGAMENTO_CONTA — QUITA — CONTA_PAGAR como N:1 e a seção 14 escreve Conta_Pagar – Pagamento_Conta como 1:N. É o mesmo relacionamento; padronizar a redação.
7. **Regras inferidas.** 'dinheiro' só na loja física, entrada não passa do comprado, soma das parcelas igual ao total da compra e lote não entra vencido não estão escritas como regra. Parecem coerentes com os processos e cenários, mas precisam de confirmação.
8. **Listas abertas.** cargo e forma_pagamento de pagamento_conta usam "Ex." no dicionário. Os CHECKs fechados protegem o dado, e novos valores entram alterando o CHECK.
9. **LGPD e exclusão.** ON DELETE RESTRICT e o histórico de 5 anos (RNF03) conflitam com pedido de exclusão do titular. A saída é anonimizar nome, telefone e e-mail do cliente, mantendo os pedidos. Como cpf é NN e UK, a anonimização troca o CPF por um valor irreversível único.

10. Índices além de PK e UK

O PostgreSQL não cria índice automático para FK, então os principais são declarados:

- movimentacao_estoque (id_lote, tipo): cálculo de v_lote_saldo e a meta de 2 s do RNF04.
- movimentacao_estoque (id_pedido, id_produto) e (id_compra, id_produto): histórico por item (RN20), quantidade recebida e recall de lote (cenário 2).
- lote (id_produto, data_validade): escolha do lote que vence antes e rotina de vencidos (RF11, RP09).
- pedido (id_cliente, data_pedido): histórico do cliente (RF27). pedido (canal_venda, data_pedido): relatórios por canal (RF31).
- pagamento (id_pedido); conta_receber (status, data_prevista): fechamento e fluxo de caixa (RF26, RF37).
- conta_pagar (status, data_vencimento) e pagamento_conta (id_conta_pagar): contas a pagar por vencimento (RF37).
- compra (id_fornecedor) e produto (id_categoria).

11. Notação relacional resumida

PK em negrito, # indica FK, UK única. 16 tabelas.

- cliente (**id_cliente**, nome, cpf UK, telefone, email)
- funcionario (**id_funcionario**, nome, cpf UK, cargo, login UK, senha)
- categoria (**id_categoria**, nome UK, descricao)
- produto (**id_produto**, nome, descricao, marca, codigo_barras UK, preco, estoque_minimo, controla_validade, #id_categoria)
- lote (**id_lote**, #id_produto, codigo_lote, data_validade, data_entrada; UK (id_produto, codigo_lote); UK (id_lote, id_produto))
- movimentacao_estoque (**id_movimentacao**, #id_lote, #id_produto, tipo, quantidade, data_hora, motivo, #id_funcionario, #id_autorizador, #id_pedido, #id_compra; FKs compostas (id_lote, id_produto), (id_pedido, id_produto), (id_compra, id_produto))
- fornecedor (**id_fornecedor**, razao_social, cnpj UK, telefone, email)
- compra (**id_compra**, #id_fornecedor, data_compra, status)
- item_compra (**#id_compra, #id_produto**, quantidade_comprada, custo_unitario)
- conta_pagar (**id_conta_pagar**, #id_compra, numero_parcela, valor, data_vencimento, status; UK (id_compra, numero_parcela))
- pagamento_conta (**id_pagamento_conta**, #id_conta_pagar, valor_pago, data_pagamento, forma_pagamento)
- pedido (**id_pedido**, #id_cliente, #id_funcionario, data_pedido, canal_venda, tipo_recebimento, status, desconto, #id_autorizador_desconto, #id_autorizador_cancelamento)
- item_pedido (**#id_pedido, #id_produto**, quantidade, preco_unitario)
- pagamento (**id_pagamento**, #id_pedido, forma_pagamento, valor, numero_parcelas, data_autorizacao, status)
- conta_receber (**id_conta_receber**, #id_pagamento, numero_parcela, valor_bruto, valor_liquido, data_prevista, data_recebimento, status; UK (id_pagamento, numero_parcela))
- entrega (**id_entrega**, #id_pedido UK, logradouro, numero, complemento, bairro, cidade, uf, cep, data_prevista, data_entrega, status)

Próximos passos sugeridos: conferir a normalização (atenção ao [A1]), gerar o modelo físico e rodar os cenários 1 a 8 como casos de teste contra as restrições e gatilhos acima.